

TFG-esports-calendar

Aplicación de calendario de eventos competitivos de videojuegos

Análisis del entorno y requisitos

El sector de los esports y los videojuegos competitivos ha crecido en los últimos años y existen múltiples plataformas que muestran eventos, pero muchas de ellas no centralizan bien la información, o quizá no ofrecen notificaciones personalizadas o tienen un seguimiento sencillo desde móvil.

Esta aplicación nace con el objetivo de centralizar eventos competitivos de videojuegos y permitir al usuario recibir notificaciones relevantes antes de que ocurran, además de gestionar sus preferencias personales y anotaciones personalizadas.

Tecnologías utilizadas

Android Studio

Backend: Django + Django REST Framework

Autenticación: JWT (JSON Web Tokens) con simplejwt

Base de datos: SQLite

API externa: PandaScore API

Notificaciones: alarmas locales de alta precisión en el dispositivo.

Diseño del prototipo: Figma

Público objetivo (Persona)

Edad: 18–30 años

Perfil: Jóvenes aficionados a los videojuegos competitivos que siguen torneos de esports (LoL, CS2, Valorant, etc.), consume contenido desde el móvil y le interesa recibir recordatorios de eventos importantes.

Dificultades y necesidades:

Poder filtrar eventos por tipo o videojuego

Recibir notificaciones antes de que empiece un evento

Gestionar preferencias personalizadas (videojuegos, equipos, ligas, torneos)

Crear anotaciones personales sobre eventos

Recibir recomendaciones basadas en sus intereses

Diseño de la aplicación

Paleta de colores y estilo

Colores principales: tonos azules degradados, junto a grises y blancos. Cada tipo de evento se identifica con un color diferente en el calendario. Diseño orientado a reducir la carga visual y que sea fácil de observar.

Figma del diseño:

<https://www.figma.com/design/ITsfracGV7VT7wmlegFu5Y/Sin-t%C3%ADtulo?node-id=0-1&t=3bEpfzfTARzqS5hB-1>

Tipografía: Inter / sans-serif moderna
Paradigma de diseño: Material Design 3

En Android uso de:

Cards para eventos
Floating Buttons para acciones principales
Bottom Navigation para navegación principal
Futura implementación de feedback
Componentes y diseño general
Pantalla de Login / Registro
Vista principal con calendario
Listado de eventos del día
Detalle del evento
Pantalla de ajustes y notificaciones
Pantalla de perfil
Gestión de anotaciones personales

Historias de usuario

Registro de usuario

Como usuario, quiero registrarme en la aplicación para guardar mis preferencias y recibir notificaciones. API: POST /api/registro/

Inicio de sesión

Para acceder a mi calendario personalizado. API: POST /api/login/

Cierre de sesión

Para cerrar mi sesión de forma segura. API: POST /api/logout/

Ver eventos en el calendario

Para ver los eventos competitivos en un calendario para saber cuándo se celebran. API: GET /api/events/

(Datos obtenidos y sincronizados desde API de PandaScore)

Sincronizar eventos

Para actualizar la base de datos local con los eventos más recientes de PandaScore. API: POST /api/events/sync/

Filtrar eventos por tipo o videojuego

Filtrar los eventos por videojuego o categoría para encontrar los que me interesan. API: GET /api/events?videogame=valor&type=valor

Suscribirse a eventos

Para recibir notificaciones de un evento específico. API: POST /api/subscriptions/ API: DELETE /api/subscriptions/<evento_id>/

Gestionar preferencias

Para guardar mis filtros y gustos (videojuegos, equipos, ligas, torneos). API: GET /api/preferences/ API: POST /api/preferences/ API: PUT /api/preferences/ API: DELETE

/api/preferences/

Recibir recomendaciones

Para obtener eventos recomendados basados en mis preferencias. API: GET

/api/preferences/recommended/

Gestionar anotaciones

Para crear notas personales sobre eventos o fechas importantes. API: GET

/api/anotaciones/ API: POST /api/anotaciones/ API: PUT /api/anotaciones/ API: DELETE

/api/anotaciones/

Actualizar token de notificaciones

Para registrar el dispositivo para recibir notificaciones push. API: POST /api/update-token/

Actualizar perfil

Para modificar la información personal del usuario. API: POST /api/update-perfil/

Flujo de la aplicación

Pantalla de bienvenida

Registro / Login

Vista principal (Calendario)

Selección de evento

Detalle del evento

Activación de notificaciones (suscripción)

Gestión de preferencias

Creación de anotaciones personales

Notificaciones: El usuario puede elegir recibir notificaciones 1 hora antes y/o 5 minutos antes del evento.

Arquitectura de la aplicación

Arquitectura general

Arquitectura cliente-servidor con API REST como intermediario entre frontend y backend.

Backend (Django)

Consumo de la API de PandaScore

Persistencia de eventos relevantes

Gestión de usuarios y autenticación JWT

Gestión de suscripciones y notificaciones

Gestión de preferencias y anotaciones

Sistema de recomendaciones basado en preferencias

Exposición de endpoints REST

Servicio de notificaciones FCM

Frontend

React Native: consumo de la API REST

Android:

Activities principales

Fragments para navegación

RecyclerView para listas de eventos

CalendarView / componente personalizado de calendario

Modelo Entidad-Relación (Backend)

Entidades principales

- Usuario
 - id
 - email (único)
 - username (único)
 - is_active
 - is_staff
 - is_superuser
 - created_at
 - notification_token (token de FCM)
 - InfoPerfil
 - user (OneToOne con Usuario)
 - birthday
 - phone
 - country

Evento

- id
- external_id (ID de PandaScore)
- scheduled_at (fecha y hora del evento)
- match_name
- league_name
- tournament_name
- serie_full_name
- videogame_name
- opponents (JSONField: lista de equipos/jugadores)
- match_type
- number_of_games
- status
- results (JSONField: resultados del partido)
- winner_id
- streams (JSONField: lista de streams disponibles)
- end_at
- created_at
- updated_at

Suscripción

- id
- usuario (FK a Usuario)
- evento (FK a Evento)
- recordatorio_1_hora (boolean)
- recordatorio_5_minutos (boolean)
- created_at

Preferencia

- id

- usuario (FK a Usuario)
- tipo_preferencia (videojuego, jugador, liga, torneo)
- valor (nombre de la preferencia)
- valor_id (ID externo)
- puntaje_interes (float)
- frecuencia (int)
- total_suscripciones_usuario (int)
- created_at
- ultima_actualizacion

Anotación

- id
- usuario (FK a Usuario)
- titulo
- descripcion
- fecha_hora
- created_at

Relaciones

Usuario 1 — 1 InfoPerfil

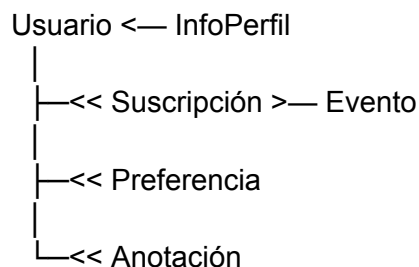
Usuario 1 — N Suscripción

Evento 1 — N Suscripción

Usuario 1 — N Preferencia

Usuario 1 — N Anotación

Diagrama simplificado:



Un evento puede estar asociado a muchos usuarios mediante suscripciones. Las preferencias permiten personalizar las recomendaciones y el filtrado de eventos. Las anotaciones permiten al usuario crear notas personales.

Este modelo entidad-relación es escalable y encaja con Django ORM para poder usar correctamente la API externa junto a la lógica propia, incluyendo sistemas de recomendación basados en las preferencias del usuario.